# La mémoire des badges

Manuel détaillé du contenu de la mémoire
Programme *Autorisation_Ouverture_Securisee_R4* · Arduino UNO R4 Minima / WiFi

### Sommaire

1. Où sont rangées les informations
2. Notions de base : octet, hexadécimal, adresse
3. Plan général de la mémoire du R4
4. L'en-tête (bloc 0) octet par octet
5. La fiche d'un badge (blocs 1 à 7)
6. Le rangement des badges dans les blocs
7. Ce que contient le badge lui-même
8. Ce qui n'est enregistré nulle part
9. Évolution de la mémoire au fil des opérations
10. Ce qui est écrit à chaque opération
11. Usure de la mémoire flash
12. Lire la mémoire avec Lecture_EEPROM_Badges_R4
13. Reconnaître les anomalies
Annexe : comparaison avec la version Uno R3

## 1. Où sont rangées les informations

Le système répartit ses informations à trois endroits différents :

| Endroit | Contenu | Durée de vie |
|---|---|---|
| **Mémoire du programme** (flash du R4, 256 Ko) | Le programme lui-même, dont la **clé maître** CLE_MAITRE (16 octets secrets) | Jusqu'au prochain téléversement |
| **EEPROM** du R4 (8 Ko simulés en flash) | La **base de badges** : badge administrateur, liste des badges, compteurs, états | Conservée après coupure de courant et après un téléversement |
| **Chaque badge** (secteur 1 de sa puce MIFARE Classic) | Sa **clé** propre, son **compteur**, son **numéro d'inscription** et une **signature** | Dans le badge, jusqu'à son retrait |

Ce manuel détaille surtout l'**EEPROM**, c'est-à-dire ce qu'affiche le programme *Lecture_EEPROM_Badges_R4*. Il s'appuie sur l'exemple réel suivant : un badge administrateur et un badge enregistré, utilisés chacun deux fois.

```
================================================
   BASE DE BADGES - UNO R4 (EEPROM simulee 8 Ko)
================================================
Capacite maximale : 791 badges
Badge ADMIN       : 86 32 3C 1F   compteur 2   inscription 0xD8D7
Nombre de badges  : 1

  No  Bloc  Adresse  UID           Passages  Inscr.  Etat
 ---  ----  -------  -----------   --------  ------  ----------------------
   1     1   0x0400  C7 FB B3 C3          2  0x7AF0  actif
================================================

Adresse  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
0x0000  B4 01 01 00 86 32 3C 1F 02 00 D7 D8 00 00 00 00
```

# 2. Notions de base : octet, hexadécimal, adresse

## L'octet

La mémoire est une longue suite de cases appelées **octets**. Chaque octet contient un nombre de **0 à 255**. La mémoire EEPROM du R4 compte 8 192 octets.

## La notation hexadécimale

Les valeurs sont écrites en **hexadécimal** (base 16), avec les chiffres 0 à 9 puis les lettres A à F (A = 10, B = 11 … F = 15). Un octet s'écrit toujours avec **deux caractères**, de 00 à FF. On ajoute parfois le préfixe **0x** pour préciser qu'il s'agit d'hexadécimal.

| Hexadécimal | Calcul | Décimal |
|---|---|---|
| 01 | 0 × 16 + 1 | 1 |
| 0A | 0 × 16 + 10 | 10 |
| 1F | 1 × 16 + 15 | 31 |
| B4 | 11 × 16 + 4 | 180 |
| FF | 15 × 16 + 15 | 255 (valeur maximale d'un octet) |

## L'adresse

Chaque octet a un numéro de position, son **adresse**, de 0x0000 à 0x1FFF (0 à 8 191). Dans le vidage, chaque ligne affiche 16 octets : l'adresse en début de ligne est celle du premier octet, et l'en-tête de colonnes 00 … 0F donne le décalage des suivants.

```
Adresse  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
0x0000  B4 01 01 00 86 32 3C 1F 02 00 D7 D8 00 00 00 00
          ^                       ^
          adresse 0x0000 = B4     adresse 0x0000 + 8 = 0x0008 = 02
```

## Les nombres sur deux octets : poids faible en premier

Un octet ne dépasse pas 255. Les compteurs, le nombre de badges et les numéros d'inscription occupent donc **deux octets** (jusqu'à 65 535). Ils sont rangés **octet de poids faible en premier** (convention dite « petit-boutiste », ou *little-endian*). Pour les lire, on inverse les deux octets :

| En mémoire | On inverse | Valeur | Exemple tiré de votre mémoire |
|---|---|---|---|
| 01 00 | 0x0001 | 1 | Nombre de badges |
| 02 00 | 0x0002 | 2 | Compteur de l'administrateur et du badge |
| D7 D8 | 0xD8D7 | 55 511 | Numéro d'inscription de l'administrateur |
| F0 7A | 0x7AF0 | 31 472 | Numéro d'inscription du badge |
| 2C 01 | 0x012C | 300 | Ce qu'on verrait avec 300 badges |

**À savoir :** le programme de lecture fait cette inversion pour vous : il affiche directement « compteur 2 » et « inscription 0xD8D7 ». Elle n'est utile que pour lire le vidage brut.

## 3. Plan général de la mémoire du R4

Le R4 n'a pas de véritable EEPROM : sa bibliothèque EEPROM utilise **8 Ko de mémoire flash de données**, découpés en **8 blocs de 1 024 octets**. Le programme les organise ainsi :

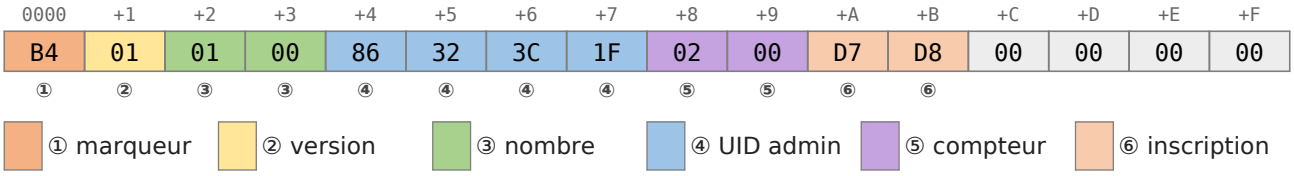
| Bloc | Adresses | Contenu | Dans votre exemple |
|---|---|---|---|
| 0 | 0x0000 – 0x03FF | En-tête : marqueur, nombre de badges, badge administrateur (12 octets utilisés sur 1 024) | B4 01 01 00 86 32 3C 1F 02 00 D7 D8 |
| 1 | 0x0400 – 0x07FF | Fiches de badges (113 au maximum) | Badge n° 1 en 0x0400 |
| 2 | 0x0800 – 0x0BFF | Fiches de badges (113 au maximum) | vide |
| 3 | 0x0C00 – 0x0FFF | Fiches de badges (113 au maximum) | vide |
| 4 | 0x1000 – 0x13FF | Fiches de badges (113 au maximum) | vide |
| 5 | 0x1400 – 0x17FF | Fiches de badges (113 au maximum) | vide |
| 6 | 0x1800 – 0x1BFF | Fiches de badges (113 au maximum) | vide |
| 7 | 0x1C00 – 0x1FFF | Fiches de badges (113 au maximum) | vide |

- **Pourquoi des blocs ?** La mémoire flash ne s'efface que par bloc entier de 1 Ko. Modifier un seul octet oblige à effacer puis réécrire tout le bloc (voir chapitre 11).
- **Pourquoi l'en-tête est-il seul dans son bloc ?** Il change rarement (ajout ou retrait d'un badge, passage de l'administrateur). Les passages des badges ne l'effacent donc jamais.
- **Pourquoi 7 blocs de badges ?** Pour répartir l'usure : les badges sont rangés à tour de rôle dans les 7 blocs (chapitre 6).

**À savoir :** au démarrage, le programme recopie l'en-tête et toutes les fiches en **mémoire vive** (RAM). Les recherches se font dans cette copie, ce qui est instantané même avec 791 badges. L'EEPROM n'est relue qu'au démarrage suivant ; elle n'est écrite qu'en cas de modification.

# 4. L'en-tête (bloc 0) octet par octet

Les 12 premiers octets de la mémoire forment l'**en-tête**. Voici votre première ligne de vidage, champ par champ :



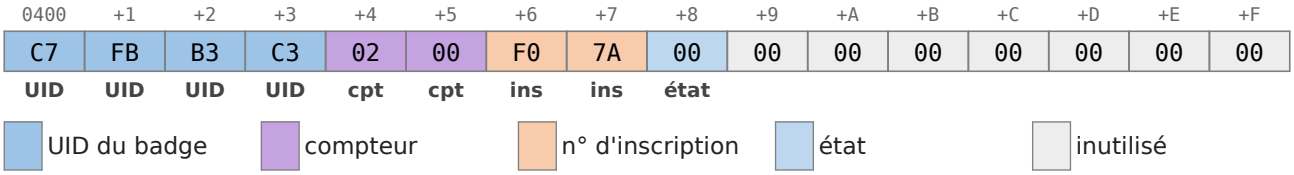

| | Adresse | Taille | Champ | En mémoire | Valeur lue |
|---|---|---|---|---|---|
| ① | 0x0000 | 1 | **Marqueur de format** : B4 signifie « base au format R4 sécurisé, badge administrateur enregistré » | B4 | base valide |
| ② | 0x0001 | 1 | **Version du format** : prévue pour de futures évolutions | 01 | version 1 |
| ③ | 0x0002 | 2 | **Nombre de badges** enregistrés, hors administrateur | 01 00 | 1 badge |
| ④ | 0x0004 | 4 | **UID du badge administrateur** : son numéro unique, recopié tel quel | 86 32 3C 1F | 86 32 3C 1F |
| ⑤ | 0x0008 | 2 | **Compteur de passages de l'administrateur** | 02 00 | 2 |
| ⑥ | 0x000A | 2 | **Numéro d'inscription de l'administrateur**, tiré au hasard lors de son enregistrement | D7 D8 | 0xD8D7 |
| | 0x000C … | 1 012 | Inutilisé (reste du bloc 0) | 00 … | — |

## Détail des champs

- ① **Marqueur B4.** Si cet octet n'est pas B4 au démarrage, le programme considère qu'il n'y a pas de base et demande un badge administrateur. Il est écrit en même temps que le reste de l'en-tête, lors de l'enregistrement de l'administrateur.
- ③ **Nombre de badges.** Il indique combien de fiches sont valides dans les blocs 1 à 7. Les fiches situées au-delà sont ignorées, même si elles contiennent encore des données. Il augmente de 1 à chaque ajout, diminue de 1 à chaque retrait, et ne peut pas dépasser 791.
- ④ **UID administrateur.** Quand un badge est présenté, son UID est comparé à celui-ci pour savoir s'il s'agit de l'administrateur.
- ⑤ **Compteur administrateur.** Il augmente de 1 à chaque passage du badge administrateur (entrée et sortie du mode programmation). Votre valeur 2 correspond à une entrée suivie d'une sortie.
- ⑥ **Numéro d'inscription administrateur.** Il est aussi écrit dans le badge administrateur. Les deux doivent être identiques ; il change si l'administrateur est réenregistré (après un RAZ).

# 5. La fiche d'un badge (blocs 1 à 7)

Chaque badge enregistré occupe une **fiche de 9 octets**. Votre badge n° 1 est rangé au tout début du bloc 1, à l'adresse **0x0400** (1 024 en décimal) :




| Décalage | Adresse | Taille | Champ | En mémoire | Valeur lue |
|---|---|---|---|---|---|
| +0 | 0x0400 | 4 | **UID du badge** | C7 FB B3 C3 | C7 FB B3 C3 |
| +4 | 0x0404 | 2 | **Compteur de passages** | 02 00 | 2 |
| +6 | 0x0406 | 2 | **Numéro d'inscription** | F0 7A | 0x7AF0 |
| +8 | 0x0408 | 1 | **État** : 00 = actif, 01 = bloqué (copie détectée) | 00 | actif |

**À savoir :** les octets qui suivent (à partir de 0x0409) appartiendront au badge n° 8, rangé lui aussi dans le bloc 1 (chapitre 6). Tant qu'il n'existe pas, ils ne sont pas utilisés et valent 00, ou FF sur une mémoire jamais effacée.

## Rôle de chaque champ

- **UID (4 octets).** Numéro unique gravé dans la puce du badge en usine. Le programme le compare à celui du badge présenté pour retrouver sa fiche. Les badges MIFARE Classic ont un UID de 4 octets.
- **Compteur (2 octets).** Nombre de passages réussis depuis l'enregistrement. La même valeur est écrite dans le badge. À chaque passage, le programme compare les deux : si le compteur du badge est **en retard**, une copie du badge a servi. Il repart de 0 après 65 535 passages, sans gêner le fonctionnement.
- **Numéro d'inscription (2 octets).** Tiré au hasard lors de l'enregistrement, et écrit aussi dans le badge. Une copie faite avant un réenregistrement porte l'ancien numéro : elle est refusée.
- **État (1 octet).** 00 en temps normal. Il passe à 01 quand une copie est détectée : le badge est alors bloqué jusqu'à sa réactivation.

| Valeur de l'état | Signification | Affichage du programme de lecture |
|---|---|---|
| 00 | Badge actif | actif |
| 01 | Badge bloqué : une copie a été détectée | BLOQUE (copie detectee) |
| autre | Valeur anormale (mémoire abîmée) | inconnu 0x.. |

# 6. Le rangement des badges dans les blocs

Les fiches ne sont pas rangées les unes à la suite des autres dans un même bloc : elles sont distribuées **à tour de rôle** dans les blocs 1 à 7. Ainsi, les passages de badges différents usent des blocs différents.

## La règle de calcul

Le programme de lecture numérote les badges à partir de 1 (colonne « No »). Pour le calcul, on utilise i = No − 1 :

```
bloc    = 1 + (i modulo 7)          (reste de la division de i par 7)
rang    = i / 7                     (division entière)
adresse = bloc x 1024 + rang x 9
```

**Votre badge n° 1** : i = 0, bloc = 1 + 0 = 1, rang = 0, adresse = 1 × 1024 + 0 = 1024 = **0x0400**. C'est bien l'adresse affichée par le programme de lecture.

**Exemple, badge n° 100** : i = 99 ; 99 = 14 × 7 + 1, donc bloc = 1 + 1 = 2 et rang = 14 ; adresse = 2 × 1024 + 14 × 9 = 2048 + 126 = 2174 = **0x087E**.

| Badge n° | Bloc | Rang | Adresse |
|---|---|---|---|
| 1 | 1 | 0 | 0x0400 |
| 2 | 2 | 0 | 0x0800 |
| 3 | 3 | 0 | 0x0C00 |
| 4 | 4 | 0 | 0x1000 |
| 5 | 5 | 0 | 0x1400 |
| 6 | 6 | 0 | 0x1800 |
| 7 | 7 | 0 | 0x1C00 |
| 8 | 1 | 1 | 0x0409 |

| Badge n° | Bloc | Rang | Adresse |
|---|---|---|---|
| 9 | 2 | 1 | 0x0809 |
| 10 | 3 | 1 | 0x0C09 |
| 11 | 4 | 1 | 0x1009 |
| 12 | 5 | 1 | 0x1409 |
| 13 | 6 | 1 | 0x1809 |
| 14 | 7 | 1 | 0x1C09 |
| 15 | 1 | 2 | 0x0412 |
| 16 | 2 | 2 | 0x0812 |

## Capacité

- Un bloc de 1 024 octets contient 1 024 / 9 = **113 fiches** (113 × 9 = 1 017 octets). Les 7 derniers octets de chaque bloc restent inutilisés : une fiche n'est jamais coupée entre deux blocs.
- 7 blocs × 113 fiches = **791 badges** au maximum.

- Le dernier badge possible, n° 791 (i = 790), est rangé dans le bloc 7 au rang 112, à l'adresse 0x1C00 + 112 × 9 = **0x1FF0**.

# 7. Ce que contient le badge lui-même

Un badge MIFARE Classic 1K possède 1 Ko de mémoire, découpé en 16 secteurs de 4 blocs de 16 octets. Le programme utilise uniquement le **secteur 1** (blocs 4 à 7). Le secteur 0, qui contient l'UID et les données du fabricant, n'est jamais modifié.

## Bloc 4 : compteur, inscription et signature

Pour votre badge C7 FB B3 C3, utilisé deux fois, le bloc 4 contient :

| 43 | 41 | 02 | 00 | F0 | 7A | ?? | ?? | ?? | ?? | ?? | ?? | ?? | ?? | 00 | 00 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 'C' | 'A' | cpt | cpt | ins | ins | sig | sig | sig | sig | sig | sig | sig | sig | | |

| Octets | Contenu | Votre badge |
|---|---|---|
| 0 – 1 | Repère **'C' 'A'** (codes 0x43 et 0x41, pour « Contrôle d'Accès ») | 43 41 |
| 2 – 3 | **Compteur** de passages (poids faible en premier) | 02 00 = 2, comme dans la fiche |
| 4 – 5 | **Numéro d'inscription** | F0 7A = 0x7AF0, comme dans la fiche |
| 6 – 13 | **Signature** de 8 octets, calculée à partir de la clé maître, de l'UID, du compteur et du numéro d'inscription | inconnue (secrète) |
| 14 – 15 | Inutilisés | 00 00 |

La signature empêche de fabriquer un faux bloc : sans la clé maître, impossible de calculer la bonne signature pour un compteur modifié. Elle change à chaque passage, puisque le compteur change.

## Bloc 7 : le verrou du secteur

| ?? | ?? | ?? | ?? | ?? | ?? | FF | 07 | 80 | 69 | ?? | ?? | ?? | ?? | ?? | ?? |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| clé A | clé A | clé A | clé A | clé A | clé A | droits | droits | droits | droits | clé B | clé B | clé B | clé B | clé B | clé B |

- **Clé A (6 octets)** : propre à chaque badge, calculée à partir de son UID et de la clé maître. Elle est nécessaire pour lire ou écrire le bloc 4. Un badge neuf a la clé FF FF FF FF FF FF.
- **Droits d'accès FF 07 80 69** : configuration standard, qui autorise la lecture et l'écriture avec la clé A.
- **Clé B (6 octets)** : seconde clé propre au badge, calculée de la même façon.

> **À savoir :** une fois la clé installée, le bloc 7 ne se relit pas en clair : la puce masque la clé A. C'est pour cela qu'un copieur ordinaire, qui ne connaît pas la clé, ne peut pas lire le bloc 4.

### Correspondance entre le badge et la mémoire de l'Arduino

| Information | Dans le badge (bloc 4) | Dans l'EEPROM (fiche) | Contrôle au passage |
|---|---|---|---|
| UID | secteur 0 (usine) | 0x0400 : C7 FB B3 C3 | identifie la fiche |
| Compteur | 02 00 | 0x0404 : 02 00 | le badge ne doit pas être en retard |
| N° d'inscription | F0 7A | 0x0406 : F0 7A | doivent être identiques |
| Signature | 8 octets | non stockée | recalculée et comparée |

## 8. Ce qui n'est enregistré nulle part

Certaines informations ne sont stockées ni dans l'EEPROM ni dans le badge. Elles sont **recalculées à chaque passage** :

| Information | Calculée à partir de | Pourquoi elle n'est pas stockée |
|---|---|---|
| Clés A et B de chaque badge | Clé maître + UID du badge (calcul SipHash) | Lire l'EEPROM ne suffit pas pour obtenir les clés des badges |
| Signature attendue | Clé maître + UID + compteur + n° d'inscription | Elle sert à vérifier celle du badge |

Seule la **clé maître**, dans le programme, permet ces calculs. C'est pourquoi il faut la garder secrète, et la reprendre à l'identique en cas de changement de carte.

## 9. Évolution de la mémoire au fil des opérations

Voici comment la mémoire a évolué pour aboutir à votre exemple. Seuls les octets utiles sont représentés.

### Étape 1 : enregistrement de l'administrateur

L'en-tête est écrit en une seule fois : marqueur B4, nombre de badges 0, UID de l'administrateur, compteur 0 et numéro d'inscription tiré au hasard (ici 0xD8D7).

| 0000 | +1 | +2 | +3 | +4 | +5 | +6 | +7 | +8 | +9 | +A | +B | +C | +D | +E | +F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| B4 | 01 | 00 | 00 | 86 | 32 | 3C | 1F | 00 | 00 | D7 | D8 | 00 | 00 | 00 | 00 |
| ① | ② | ③ | ③ | ④ | ④ | ④ | ④ | ⑤ | ⑤ | ⑥ | ⑥ | | | | |

### Étape 2 : passage de l'administrateur (entrée en mode programmation)

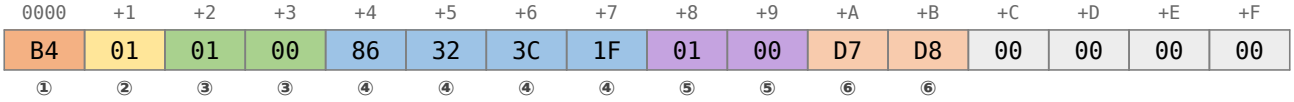
Le compteur de l'administrateur passe à 1, dans le badge comme dans l'en-tête.

| 0000 | +1 | +2 | +3 | +4 | +5 | +6 | +7 | +8 | +9 | +A | +B | +C | +D | +E | +F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| B4 | 01 | 00 | 00 | 86 | 32 | 3C | 1F | 01 | 00 | D7 | D8 | 00 | 00 | 00 | 00 |
| ① | ② | ③ | ③ | ④ | ④ | ④ | ④ | ⑤ | ⑤ | ⑥ | ⑥ | | | | |

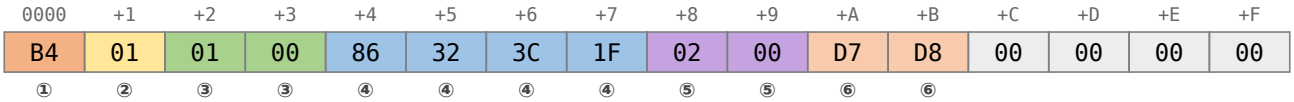
### Étape 3 : enregistrement du badge C7 FB B3 C3

La fiche n° 1 est écrite en 0x0400 avec le compteur 0, le numéro d'inscription tiré au hasard (0x7AF0) et l'état actif. Puis le nombre de badges passe à 1 dans l'en-tête.

| 0400 | +1 | +2 | +3 | +4 | +5 | +6 | +7 | +8 |
|---|---|---|---|---|---|---|---|---|
| C7 | FB | B3 | C3 | 00 | 00 | F0 | 7A | 00 |
| **UID** | **UID** | **UID** | **UID** | **cpt** | **cpt** | **ins** | **ins** | **état** |

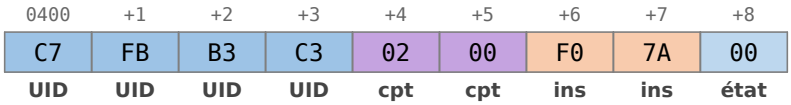


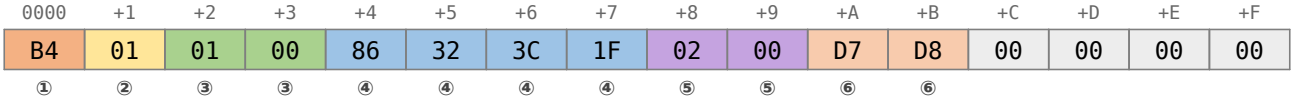

## Étape 4 : passage de l'administrateur (sortie du mode programmation)




## Étapes 5 et 6 : deux passages du badge

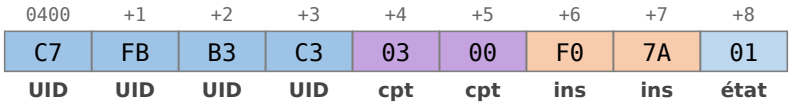
À chaque ouverture, le compteur augmente de 1 dans le badge puis dans la fiche : 00 00 › 01 00 › **02 00**. On obtient exactement votre mémoire :



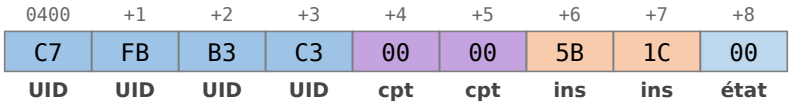

## Si une copie est détectée

Supposons qu'une copie du badge, faite quand son compteur valait 2, soit présentée après un troisième passage du vrai badge. La fiche contient 03 00 et la copie présente 2 : elle est en retard. L'état passe à 01 :




## Après une réactivation

Le badge est retiré puis réenregistré : nouvelle fiche, compteur 0, état actif et **nouveau numéro d'inscription** (par exemple 0x1C5B). La copie, qui porte encore 0x7AF0, sera refusée.




## Retrait d'un badge : le dernier prend sa place

Avec trois badges A (n° 1, 0x0400), B (n° 2, 0x0800) et C (n° 3, 0x0C00), le retrait de A recopie la fiche de C en 0x0400, puis le nombre de badges passe de 3 à 2. B n'est pas déplacé.

| | Fiche n° 1 (0x0400) | Fiche n° 2 (0x0800) | Fiche n° 3 (0x0C00) | Nombre |
|---|---|---|---|---|
| Avant | A | B | C | 03 00 |
| Après | C (recopié) | B | C (ancienne copie, ignorée) | 02 00 |

**À savoir :** la fiche 0x0C00 contient encore les données de C, mais elle se trouve au-delà du nombre de badges : elle est ignorée, et sera écrasée par le prochain badge enregistré. Ce procédé évite de décaler toutes les fiches, ce qui userait inutilement la mémoire.

### Effacement complet (RAZ)

Les 8 blocs sont remplis de 00. Le marqueur n'étant plus B4, le programme demande un nouveau badge administrateur au démarrage suivant.

| 0000 | +1 | +2 | +3 | +4 | +5 | +6 | +7 | +8 | +9 | +A | +B | +C | +D | +E | +F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

## 10. Ce qui est écrit à chaque opération

| Opération | Écrit dans le badge | Écrit dans l'EEPROM | Blocs effacés |
|---|---|---|---|
| Passage d'un badge autorisé | Bloc 4 : compteur + 1, nouvelle signature | Compteur de la fiche (2 octets) | 1 (bloc de la fiche) |
| Passage refusé (inconnu, faux, lecture incomplète) | rien | rien | 0 |
| Copie détectée | rien | État de la fiche › 01 | 1 |
| Passage de l'administrateur | Bloc 4 : compteur + 1 | En-tête (compteur administrateur) | 1 (bloc 0) |
| Enregistrement d'un badge | Bloc 4 (compteur 0, inscription) puis bloc 7 (clés) | Fiche complète, puis en-tête (nombre) | 2 |
| Retrait d'un badge | Bloc 4 à zéro, clés d'usine dans le bloc 7 | Fiche du dernier badge recopiée, puis en-tête | 1 ou 2 |
| Enregistrement de l'administrateur | Bloc 4 puis bloc 7 | En-tête complet | 1 |
| Effacement complet (RAZ) | rien (les badges gardent leurs clés) | Les 8 blocs à 00 | 8 |

**À savoir :** l'ordre des écritures protège contre les coupures de courant : le badge est écrit avant l'EEPROM, et le nombre de badges est mis à jour en dernier. Si le courant est coupé entre les deux, le badge est simplement « en avance » d'un passage, ce que le programme tolère (jusqu'à 3).

## 11. Usure de la mémoire flash

Un bloc de mémoire flash supporte environ **100 000 effacements**. Le programme les économise :

- les octets modifiés en même temps sont écrits en une seule opération : **un effacement par passage** ;
- les badges sont répartis sur 7 blocs : avec plusieurs badges, chaque bloc ne reçoit qu'environ 1/7 des passages ;
- un retrait ne recopie qu'une seule fiche ;
- l'EEPROM n'est jamais réécrite si rien ne change (badge refusé, simple lecture).

| Situation | Durée de vie estimée |
|---|---|
| Plusieurs badges bien répartis, 10 passages par jour au total | 7 × 100 000 = 700 000 passages, soit plus de 150 ans |
| Un seul badge utilisé, 10 passages par jour | 100 000 passages sur son bloc, soit plus de 27 ans |

| Situation | Durée de vie estimée |
|---|---|
| Badge administrateur | Bloc 0 : utilisé seulement en programmation, usure négligeable |

## 12. Lire la mémoire avec Lecture_EEPROM_Badges_R4

Ce programme lit l'EEPROM **sans rien modifier**. Téléversez-le, ouvrez le moniteur série à **115200 bauds**, puis retéléversez le programme de contrôle d'accès : la base est conservée.

| Commande | Effet |
|---|---|
| r | Relit et affiche la base décodée (comme l'exemple du chapitre 1) |
| d | Vidage brut du bloc 0 (1 024 octets, 64 lignes) : l'en-tête est sur la première ligne |
| D | Vidage brut complet des 8 Ko (512 lignes) : pour retrouver une fiche à son adresse |

Pour retrouver votre badge dans le vidage complet, cherchez la ligne **0x0400** :

```
Adresse  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
0x0400  C7 FB B3 C3 02 00 F0 7A 00 00 00 00 00 00 00 00
        |---UID---| |cpt| |ins| |
                              etat
```

**Astuce :** le vidage complet est long. Pour aller directement à une fiche, calculez son adresse avec la règle du chapitre 6, puis cherchez la ligne qui commence par cette adresse arrondie à la dizaine hexadécimale inférieure (par exemple 0x0870 pour une fiche en 0x087E).

## 13. Reconnaître les anomalies

| Ce que l'on voit | Signification | Que faire |
|---|---|---|
| Tous les octets à FF | Mémoire neuve, jamais utilisée | Rien : enregistrer l'administrateur |
| Tous les octets à 00 | Mémoire effacée par un RAZ | Rien : enregistrer l'administrateur |
| Premier octet différent de B4 | Pas de base valide au format R4 | Le programme demande un administrateur |
| Nombre de badges supérieur à 791 | En-tête abîmé | Remis à 0 automatiquement au démarrage |
| État 01 dans une fiche | Badge bloqué : une copie a servi | Réactiver le badge en mode programmation |
| Deux fiches avec le même UID | Coupure de courant pendant un retrait (rarissime) | Retirer le badge, puis le réenregistrer |
| Fiche identique à l'UID administrateur | Anomalie de saisie | Effacer la base (RAZ) et tout réenregistrer |

# Annexe : comparaison avec la version Uno R3

La version pour Uno R3 utilise une vraie EEPROM de 1 Ko, que l'on peut modifier octet par octet. Son organisation est plus simple :

| | Version Uno R3 | Version UNO R4 |
|---|---|---|
| Mémoire | EEPROM de 1 Ko, octet par octet | 8 Ko simulés en flash, par blocs de 1 Ko |
| Marqueur | 0xA5 à l'adresse 0x01 | 0xB4 à l'adresse 0x0000 |
| Nombre de badges | 1 octet en 0x00 | 2 octets en 0x0002 |
| Administrateur | UID 0x02, compteur 0x06, inscription 0x08 | UID 0x0004, compteur 0x0008, inscription 0x000A |
| Fiches de badges | À la suite à partir de 0x10, 9 octets | Réparties sur les blocs 1 à 7, 9 octets |
| Contenu d'une fiche | UID + compteur + inscription + état | Identique |
| Capacité | 112 badges | 791 badges |
| Retrait | Les fiches suivantes sont décalées | Le dernier badge prend la place |
| Contenu du badge | Secteur 1, blocs 4 et 7 | Identique |

**À savoir :** le contenu des badges est le même dans les deux versions : avec la même clé maître, un badge préparé sur l'Uno est reconnu sur le R4 après réenregistrement, et inversement. Seule la base de l'Arduino doit être recréée.